

ATIVIDADE PRÁTICA

Views Materialized Views e Estratégias de Indexação no Oracle

Situação-problema, exemplos executáveis, experimentação e exercícios

Disciplina	Ambiente	Tempo sugerido
Laboratório de Banco de Dados	Oracle Database 19c/21c	2 horas

Professor: Prof. Dr. Bruno Aguilar da Cunha

Nomes: Alfred Domke Albach - César Augusto Guzmán Fogaça de Almeida - Davi Cwen Shi Guo - Murilo Ferrari Menezes

Turma: 4º ADS Matutino

Data: 24 / 09 / 2026

Objetivo geral. Compreender quando utilizar views lógicas e materializadas, como aplicar segurança e consolidação analítica e onde a indexação física realmente pode ser criada.

EXERCÍCIO PRÁTICO 1 VIEW E ÍNDICE

Objetivo: criar uma camada segura para acompanhamento de pedidos ainda não finalizados e propor o índice físico adequado.

Requisitos da view VW_PEDIDOS_ATIVOS:

- Exibir pedido, data, valor, status, cliente, telefone mascarado e região.
- Mostrar somente pedidos ABERTO ou EM_ROTA e usar WITH READ ONLY.
- Criar um índice na tabela PEDIDO para STATUS e DATA_PEDIDO.
- Gerar o plano da consulta e indicar se o índice foi utilizado.

Consulta que deve ser atendida:

```
SELECT id_pedido, data_pedido, valor_total, status
FROM vw_pedidos_ativos
WHERE status = 'EM_ROTA'
AND data_pedido BETWEEN DATE '2026-09-01' AND DATE '2026-09-30';
```

SCRIPT USADO:

```
create or replace view ex1 as
select e.id_pedido, p.data_pedido, p.valor_total, p.status, c.nome, '*****' ||
SUBSTR(c.telefone,-4) AS telefone_mascarado, r.nome_regiao
from entrega e
JOIN pedido p ON p.id_pedido = e.id_pedido
JOIN cliente c ON c.id_cliente = p.id_cliente
JOIN regiao r ON r.id_regiao = p.id_regiao
WHERE e.status IN ('ABERTO','EM_ROTA')
WITH CHECK OPTION;

SELECT id_pedido, data_pedido, valor_total, status
FROM ex1
WHERE status = 'EM_ROTA'
AND data_pedido BETWEEN DATE '2026-09-01' AND DATE '2026-09-30';
```

1. Por que o índice foi criado em PEDIDO e não diretamente na view? O plano utilizou esse índice?

As views criadas ficam armazenadas na memória principal (RAM) e não na secundária, não é possível criar o índice nela. Já como uma *Materialized View* armazena os dados fisicamente no disco, ela permite a criação de índices nela mesma para acelerar leituras.

Como o script executa um **SELECT *** sem nenhuma cláusula **WHERE** para filtrar datas ou regiões específicas, o banco de dados precisa ler todas as linhas. Nesse cenário, o otimizador prefere varrer a tabela inteira de uma vez em vez de gastar tempo consultando o índice.

EXERCÍCIO PRÁTICO 2 MATERIALIZED VIEW

Objetivo: criar um resumo analítico diário de faturamento por região e estado do pedido.

Requisitos:

- Criar o Materialized View Log necessário na tabela PEDIDO.
- Criar MV_FATURAMENTO_DIARIO por data, região e status, com COUNT e SUM.
- Usar REFRESH FAST ON COMMIT e criar um índice para data e região.

- Inserir um pedido e comparar a MView antes e depois do COMMIT.

```
INSERT INTO pedido
(id_cliente, id_regiao, data_pedido, valor_total, status)
VALUES (1, 10, DATE '2026-09-22', 180.00, 'ENTREGUE');
```

```
-- Consulte a MView antes e depois deste comando.
COMMIT;
```

SCRIPT USADO:

```
-- 1. Criar o Materialized View Log
CREATE MATERIALIZED VIEW LOG ON pedido
WITH SEQUENCE, ROWID
(id_regiao, data_pedido, status, valor_total)
INCLUDING NEW VALUES;

-- 2. Criar a Materialized View
CREATE MATERIALIZED VIEW mv_faturamento_diario
BUILD IMMEDIATE
REFRESH FAST ON COMMIT
AS
SELECT
TRUNC(data_pedido) AS data_pedido,
id_regiao,
status,
COUNT(*) AS quantidade_pedidos,
COUNT(valor_total) AS quantidade_valores,
SUM(valor_total) AS faturamento_total
FROM pedido
GROUP BY
TRUNC(data_pedido),
id_regiao,
status;
```

```

-- 3. Criar índice para data e região
CREATE INDEX idx_mv_faturamento_data_regiao
ON mv_faturamento_diario (data_pedido, id_regiao);

-- 4. Consultar a MV ANTES do novo pedido
SELECT *
FROM mv_faturamento_diario
ORDER BY data_pedido, id_regiao, status;

-- 5. Inserir novo pedido
INSERT INTO pedido
(id_cliente, id_regiao, data_pedido, valor_total, status)
VALUES
(1, 10, DATE '2026-09-22', 180.00, 'ENTREGUE');

-- 6. Consultar ANTES do COMMIT
SELECT *
FROM mv_faturamento_diario
ORDER BY data_pedido, id_regiao, status;

-- 7. Confirmar a transação
COMMIT;

-- 8. Consultar DEPOIS do COMMIT
SELECT *
FROM mv_faturamento_diario
ORDER BY data_pedido, id_regiao, status;

```

2. O que mudou depois do COMMIT e qual custo o ON COMMIT pode acrescentar ao ambiente OLTP?

O ponto mais importante é entender que o ON COMMIT faz a Materialized View ser atualizada automaticamente no momento do COMMIT, e não no momento do INSERT. A própria documentação da Oracle confirma esse comportamento e também alerta que isso pode aumentar o tempo do COMMIT.

Antes do COMMIT, o novo pedido ainda não estava refletido na Materialized View. Após o COMMIT, a Materialized View foi atualizada automaticamente pelo FAST REFRESH, passando a apresentar o novo registro de faturamento de R\$ 180,00 para a região 10, com status ENTREGUE.

O uso de ON COMMIT pode aumentar o tempo necessário para concluir o COMMIT, pois o banco precisa executar a atualização da Materialized View como parte da própria operação de confirmação da transação. Em um ambiente OLTP com muitas operações simultâneas de INSERT, UPDATE e DELETE, esse processamento adicional pode aumentar o custo das transações e o uso de recursos do banco.

EXERCÍCIOS TEÓRICOS

1. Explique por que uma view convencional não reduz necessariamente o número de blocos lidos em relação à consulta original.

R: Uma view convencional não reduz necessariamente o número de blocos lidos em relação à consulta original, pois a view não armazena dados fisicamente, mas sim digitalmente, além de que a consulta convencional é feita por trás da view quando criada e executada.

2. Qual é a função de WITH CHECK OPTION? Em que situação o ORA-01402 é gerado?

R: A função do WITH CHECK OPTION é de garantir que toda a ação de atualizar uma view (insert ou update) respeite o filtro estabelecido por condições pré-estabelecidas como a cláusula WHERE.

3. Por que o Oracle pode ignorar um índice existente e escolher Full Table Scan?

R: Porque o otimizador da aplicação de banco de dados pode optar pelo Full Table Scan no momento em que percebe que a pesquisa será mais eficiente e rápida que usar índice, nesse caso.

4. Compare uma VIEW convencional com uma MATERIALIZED VIEW quanto a armazenamento, atualização e indexação.

R: Uma view convencional, por não armazenar dados fisicamente, não ocupa espaço de armazenamento, está sempre atualizada e não pode ser indexada livremente (por ser uma tabela digital, não consegue armazenar dados fisicamente). Do outro lado, uma materialized view ocupa espaço no armazenamento já que armazena dados fisicamente como uma tabela, é necessário atualizar a view materializada e, ao contrário da view convencional, a materializada por ser indexada livremente, já que não tem as limitações digitais da outra view.

5. Compare REFRESH COMPLETE e FAST e explique a diferença entre ON COMMIT e ON DEMAND.

R: As duas funcionam em views materializadas, porém a Refresh Complete reconstrói a visão do zero e a fast, a partir das últimas atualizações de insert, update e delete. A diferença entre On Commit e On Demand é que On Commit atualiza a view quando a tabela é atualizada e a On Demand, apenas quando o usuário atualiza explicitamente por meio de comando.

CONCLUSÃO COMPARANDO VIEW E MATERIALIZED VIEW

A VIEW e a MATERIALIZED VIEW permitem apresentar dados de uma ou mais tabelas de forma organizada, mas possuem comportamentos diferentes. A VIEW armazena apenas a consulta SQL, portanto os dados são obtidos das tabelas originais no momento em que a consulta é realizada. Isso permite que a view apresente dados atualizados, sendo útil para consultas, organização e controle de acesso.

Já a MATERIALIZED VIEW armazena fisicamente o resultado da consulta. Dessa forma, consultas analíticas e agregações podem ser realizadas sobre os dados já materializados, reduzindo a necessidade de recalcular toda a consulta a cada acesso. Porém, os dados precisam ser atualizados por meio de um processo de refresh. No exercício realizado, foi utilizado REFRESH FAST ON COMMIT, fazendo com que a materialized view fosse atualizada após o COMMIT.

Em resumo, a VIEW é indicada principalmente para organizar consultas e controlar o acesso aos dados, enquanto a MATERIALIZED VIEW é voltada ao armazenamento prévio dos resultados, podendo melhorar o desempenho de consultas analíticas, mas exigindo espaço adicional e atualização dos dados.